

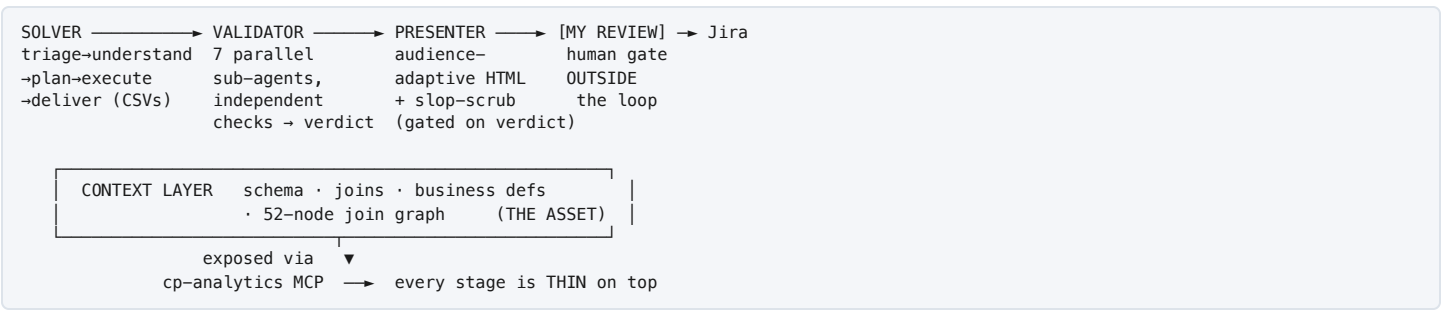
Autonomous Jira Ticket Pipeline

LEAD BUILD #1

A **3-stage autonomous pipeline** — solve → validate → present — that takes a BDV Jira ticket from raw text to a validated, stakeholder-ready deliverable with **zero internal checkpoints**.

400+ tickets since Jan '25 · **30–60 min** → **~10 min** · ~17–60 workdays saved

THE ARCHITECTURE — 3 SKILLS ON ONE REUSABLE SUBSTRATE



FOLDER CONTRACT — READ-ONLY INPUTS FLOW DOWNSTREAM, EACH STAGE APPENDS

tickets/BDV-XXXX/ ticket.json · ticket.md · attachments/ [input, read-only]

Solver → **worklog.md** + **results/** (manifest.json · QN.sql · QN.csv · QN.meta.json) Validator → **validation/report.md** + 7 check files Presenter → **deliverable/** (README · ask-coverage · artifacts/; versioned -v2, -v3, never overwritten)

1 · SOLVER — resolve

WHAT

- 4 phases: triage (6 gates) → understand → plan → execute → deliver
- MCP for every business def — **never guesses**; no SQL before MCP

SELF-HEALING

- 10-retry budget (3 syntax · 2 timeout · 3 zero-row), capped
- Zero-row → Investigation Branch: hypothesis → 5-min probe → apply

STATE

- manifest.json (machine truth) + worklog.md (narrative) → crash & resume, no re-runs

GATE

Validator gates the presenter; I gate the final send. Today I'm the gate → next: golden-set regression.

2 · VALIDATOR — verify

WHAT

- Pure dispatcher — runs no SQL itself; launches **7 parallel sub-agents**, each an independent check (the eval, productized)

THE 7 CHECKS

- Completeness — WHO/WHAT/WHEN answered?
- Definitions vs MCP **★ most critical**
- SQL quality — cost-safe, no anti-patterns
- Result verification — CSVs re-run vs live source
- Worklog quality — documentation useful?
- End-to-end coherence — one story?
- Process analysis — was MCP called before SQL?

VERDICT

- PASS / WARN / FAIL — gates the presenter

ABSTAIN

Grounded — cites only data the SQL returned. Budget 0 → partial + logged blockers, never a fabricated number.

3 · PRESENTER — package

WHAT

- Turns a validated result into a stakeholder deliverable — self-contained HTML

DECISIONS

- Validation gate**: refuses to build on a FAIL / stale result unless forced
- Re-extracts the **literal ask** from the ticket — not the solver's spin (drift check)
- Audience-adaptive: business vs engineer tone auto-detected

THEN

- Slop-scrub sub-agent strips AI filler; never overwrites originals

FEAR

Confident-wrong — clean SQL on a stale flag. Hence freshness stamps (frozen 30d / late 24h / streaming 1h) + the validator's source re-run.

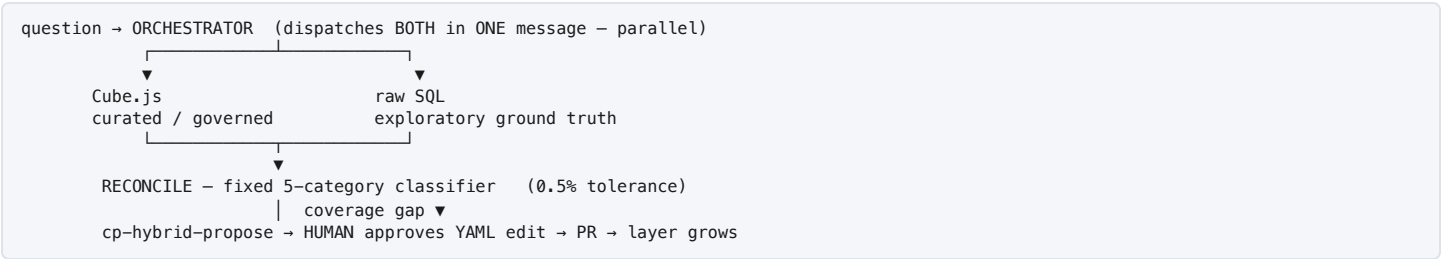
Leverage close: "The intake bottleneck was what kept me from building anything else. Automating my own role bought me the time to build the rest of the portfolio."

Autonomy posture: agent autonomous to a validated deliverable; my review sits OUTSIDE the loop — two failure budgets, each sized to blast radius. | If "draw it": 3 stages + substrate underneath, 60 sec, narrate every box. Pick 2–3 numbers max.

Dual-backend analytics orchestrator — runs the **Cube.js semantic layer** and **raw BigQuery SQL in parallel**, reconciles the two answers, and turns every disagreement into labeled data for the semantic layer.

Don't volunteer — wins the room **if** Maan probes "how do you trust a metric?" or asks about the semantic layer.

THE ARCHITECTURE — FORK, RECONCILE, SELF-GROW



RECONCILIATION LOGIC — THE FIXED 5-CATEGORY CLASSIFIER

CATEGORY	CUBE	RAW	ACTION
SAME_INTENT SAME_RESULT	ok	ok ≈	Present one answer — "✓verified by both paths." The high-trust case.
SAME_INTENT DIFFERENT_RESULT	ok	ok ≠	Present both + a diagnostic checklist (filters · joins · grain · nulls). The divergence is the signal — it exposes a modeling bug.
AMBIGUOUS QUESTION	ok	ok ≠	Present both + ask the user to clarify intent — don't silently pick one.
COVERAGE_GAP	error	ok	Cube can't answer → present raw as "△exploratory." Routes the gap to cp-hybrid-propose → the self-grow loop.
LLM_FAILURE	error	error	Surface both errors, ask to retry. Never fabricates a reconciled number.

DESIGN DECISIONS — LEAD WITH THE ARCHITECTURAL REASON

- Why two paths?** Semantic layer = curated/governed; raw SQL = exploratory truth. A disagreement *exposes a modeling bug* instead of silently shipping a wrong number. The divergence is the signal.
- Fixed 5-category classifier.** Strict taxonomy keeps the orchestrator deterministic. New category? Extend the spec doc — don't let the LLM invent one mid-run. Agent where input varies, deterministic where I can verify.
- Gated auto-improve loop.** Gaps draft a Cube YAML edit, but a human approves before merge. *Capability built; deployment gated behind a flag until the org trusts the loop.*
- Durable-local + best-effort mirror.** File-locked JSONL is canonical (atomic, survives network loss); BQ mirror adds cross-session observability without coupling UX to the network.

THE SELF-MONITORING DETAIL (LANDS WITH A PHD SCIENTIST)

- The orchestrator checks its **own** parallelism: if `cube_latency + raw_latency ≈ total` (vs `max + overhead`), dispatch went sequential → records `parallelism_degraded: true`.
- The system **measures whether it actually did the thing it claims to do**.

GATE
Human approves every layer edit; dedup ledger stops processed gaps re-surfacing.

ABSTAIN
Both paths error → `both_error`, asks retry. Ambiguous → presents both, asks intent. Never fabricates.

FEAR
Confident-wrong from a single path — a clean Cube measure on a wrong filter. The 2nd path exists to catch exactly that.

Substrate through-line: "Same context substrate as the Jira agent and the analyst — but here validation sits between two paths, and disagreements become training data. One reusable layer; this is the next thin thing on top of it."

If "draw it": the fork-and-reconcile diagram. 5 boxes, 60 sec. The money beat is the **coverage-gap → human-gated PR loop** — that's what makes it a self-growing semantic layer, not just a checker.